AMS OS

Atomic Management System

Mini Operating System Simulator

CREATORS: M Saim Naveed(24f-3043), Mohsin Ali Shah (24f-0014), Ahmed Naeem(23f-3113)

Final Project Report

Operating Systems Lab · Spring 2026
Phase 3 / Final Submission
Target Platform: Ubuntu Linux · Language: C++17 · Build: GNU Make
Generated: May 10, 2026

Table of Contents

1. Executive Summary

AMS OS (Atomic Management System) is a Linux terminal application that simulates core operating-system responsibilities. It accepts hardware resources at startup, displays a named boot screen, creates task processes using fork() and exec(), grants or denies resources through IPC pipes, tracks processes in a PCB table, and schedules ready tasks with multilevel queues. The system exposes both User Mode and Kernel Mode controls and ships with an SFML-based graphical scheduler visualizer.

The implementation is organized into kernel subsystems at the project root and independent task executables compiled into the build/ directory. Runtime state is recorded in data/system_log.txt, and file-oriented tasks operate on a virtual disk under data/virtual_disk/.

2. Project Scope

AMS OS demonstrates the following operating-system concepts:

- Multitasking through multiple child processes and multilevel ready queues
- Process creation using fork() and program replacement using exec()
- Inter-process communication using anonymous pipes (IPCResourceRequest / IPCResourceResponse)
- Resource allocation for RAM, hard drive space, and CPU cores with first-fit memory layout
- Process lifecycle management through PCB state transitions (NEW → READY → RUNNING → BLOCKED / MINIMIZED → TERMINATED)
- Context switching with scheduler dispatch and POSIX signals (SIGSTOP / SIGCONT)
- Multilevel queue scheduling with FCFS at the system and background levels and Round Robin at the interactive level
- Priority aging to prevent starvation
- Synchronization using semaphore, mutex, condition variable, and background monitor thread
- User Mode and Kernel Mode separation with password authentication
- Interrupt simulation through minimize, block, resume, and close operations
- Deadlock detection through a circular wait-for graph with DFS cycle detection
- SFML graphical scheduler terminal for visual event monitoring
- Runtime logging for all process and resource events

3. System Requirements

3.1 Target Environment

- Ubuntu 22.04 or newer (recommended)
- GNU Make
- g++ with C++17 support
- GNOME Terminal, x-terminal-emulator, or xfce4-terminal
- SFML 2.x libraries (required for the graphical scheduler terminal)

3.2 Installation Commands

```
sudo apt update
sudo apt install build-essential gnome-terminal libsfml-dev
```

Optional lightweight terminal fallback:

```
sudo apt install xfce4-terminal
```

3.3 Build and Run

```
make check
make
./OS 2 256 8
```

The command above starts AMS OS with the recommended configuration: 2 GB RAM, 256 GB hard drive, and 8 CPU cores. Task executables are compiled into build/ by the same Makefile.

4. System Architecture

The diagram below shows the high-level subsystem layout of AMS OS. All kernel modules reside at the project root alongside the Makefile. Task executables are compiled from matching .cpp source files into build/.

Kernel Subsystem Map

Layer	Module	Responsibility
Entry Point	main.cpp	Boot, menus, IPC, process controls, mode switching, shutdown
Resource	resource_manager	RAM / HDD / CPU counters and first-fit memory block allocation
Process	process_manager	PCB table and full process lifecycle management
Catalog	task_catalog	Static metadata for all 15 task executables
Queues	ready_queue	Multilevel queue: System (FCFS) / Interactive (RR) / Background (FCFS)
Dispatch	scheduler	FCFS and Round Robin dispatch, SIGSTOP / SIGCONT context switching
Sync	sync_manager	Semaphore, mutex, condition variable, background monitor thread
Deadlock	deadlock_manager	Wait-for graph construction and DFS cycle detection
Logging	logger	Timestamped event appending to data/system_log.txt
GUI	scheduler_terminal	SFML graphical window displaying scheduler event log
UI Helpers	console_colors.h	ANSI color utilities for terminal table rendering

5. Source Organization

All source files are located at the project root. The Makefile compiles the kernel executable (OS) from the kernel source files and task executables into build/. Runtime directories (data/, data/virtual_disk/) are created automatically.

Kernel Sources (project root)

main.cpp	Boot, menus, IPC, process controls
resource_manager.cpp/h	RAM, HDD, CPU counters and memory layout
process_manager.cpp/h	PCB table and lifecycle operations
task_catalog.cpp/h	Metadata for all 15 task executables
ready_queue.cpp/h	Multilevel ready queue implementation
scheduler.cpp/h	FCFS/RR scheduler and context switching
scheduler_terminal.cpp	SFML graphical scheduler event viewer
sync_manager.cpp/h	Semaphore, mutex, condition variable, thread
deadlock_manager.cpp/h	Circular wait detection
logger.cpp/h	System, process, and resource logging
console_colors.h	ANSI terminal color utilities

Task Sources → compiled to build/

calculator.cpp	delete_file.cpp	download_simulator.cpp
file_copy.cpp	file_info.cpp	minesweeper.cpp
move_file.cpp	music_player.cpp	notepad.cpp
process_killer.cpp	clock.cpp	snake.cpp
system_info.cpp	task_manager.cpp	create_file.cpp

Runtime Directories

build/	Task executable binaries
data/system_log.txt	Runtime event log
data/virtual_disk/	Virtual disk for file tasks

6. Kernel Module Design

6.1 Main Kernel Controller (main.cpp)

main.cpp controls startup, boot display, hardware resource input, menu routing, task launch, IPC, process control, mode switching, and shutdown. It creates required runtime directories using C++17 filesystem APIs. The task launch workflow is:

- User selects a task from the catalog.
- Kernel creates request and response pipes.
- Kernel forks a child process.
- Child sends an IPCResourceRequest to the kernel through the pipe.
- Kernel checks RAM, HDD, and CPU availability via the Resource Manager.
- Kernel grants or denies the request through the response pipe.
- If granted, the kernel allocates a simulated RAM block and creates a PCB entry.
- The child waits with SIGSTOP.
- The Scheduler resumes the child with SIGCONT.
- The child replaces itself with the task executable using exec().

6.2 Resource Manager

The Resource Manager tracks total and available RAM (in MB), total and available hard drive space (in MB), and total and available CPU cores. Simulated RAM is laid out as contiguous blocks with startAddress and endAddress fields stored in each PCB. When a process terminates the manager releases its block and merges adjacent free blocks to reduce fragmentation.

6.3 Process Manager

Each process is stored in a PCB record containing:

- PID, process name, process type
- Lifecycle state (NEW / READY / RUNNING / BLOCKED / MINIMIZED / TERMINATED)
- Priority, waiting time, turnaround time
- RAM, HDD, and CPU requirements
- Assigned CPU core ID
- Ready queue label
- RAM block start and end addresses

6.4 Task Catalog

The Task Catalog stores static metadata for all 15 task executables. Each entry holds task ID, name, process type, priority, RAM / HDD / CPU requirements, executable path under build/, and a description. Registration is centralized through a single helper method to enforce consistent metadata and reduce duplication.

6.5 Ready Queue Manager

AMS OS implements a three-level multilevel ready queue:

Queue	Priority	Algorithm	Assigned Tasks
System Queue	1 (highest)	FCFS	Task Manager, Process Killer
Interactive Queue	2	Round Robin (8 s quantum)	Calculator, Notepad, games, file info, Clock, Calendar
Background Queue	3	FCFS	Copy File, Move File, Music Player, Download Simulator

Priority aging is applied before each dispatch. If a process waits beyond the aging threshold its numeric priority improves and it may move to a higher-priority queue, preventing starvation.

6.6 Scheduler

The scheduler selects the next process from the System queue first, then Interactive, then Background. Context switching is demonstrated by: acquiring CPU core slots through the Sync Manager; assigning a core ID to the PCB; moving the selected process to RUNNING and sending SIGCONT; waiting for completion or time-quantum expiry; sending SIGSTOP when an Interactive process exhausts its quantum; re-queueing unfinished tasks; and releasing all resources when a task terminates.

6.7 Sync Manager

The following synchronization primitives are implemented:

Primitive	Implementation	Purpose
Semaphore (SimpleSemaphore)	mutex + condition_variable	Controls available CPU core dispatch slots
Mutex	std::mutex	Protects the CPU core ID pool
Condition variable	std::condition_variable	Notifies scheduler when a ready process exists
Thread	std::thread	Background resource monitor logs available resources periodically
Atomic flag	std::atomic<bool>	Stops the monitor thread safely on shutdown

6.8 Deadlock Manager

The deadlock module simulates circular wait between two selected processes. It builds a wait-for graph from DeadlockRecord entries (each recording PID, process name, held resource, and waited-for resource), detects cycles with DFS, and displays the required message: "Deadlock detected among processes". A victim process is then selected and terminated to recover resources.

6.9 Scheduler Terminal (SFML GUI)

scheduler_terminal.cpp implements a graphical window using the SFML library. It reads scheduler event log entries from data/scheduler_event_log.txt and renders them as a scrollable event table. This provides a visual complement to the terminal-based menus and fulfills the graphics library requirement noted as a bonus in the project manual.

6.10 Logger

The Logger appends timestamped events to data/system_log.txt. Recorded event categories include: process creation, resource allocation and release, RAM block assignment, scheduler dispatch, interrupt and state change, process termination, Kernel Mode access, deadlock detection, and shutdown cleanup.

7. Task Inventory

The following 15 tasks are registered in the Task Catalog. Each is compiled from a dedicated source file into its own executable under build/ and launched as a separate child process via exec().

ID	Task Name	Source File	Type	Pri	RAM	HDD	CPU
1	Create File	create_file.cpp	Interactive	2	80 MB	20 MB	1
2	Delete File	delete_file.cpp	Interactive	2	70 MB	10 MB	1
3	Copy File	file_copy.cpp	Background	3	200 MB	100 MB	1
4	Move File	move_file.cpp	Background	3	150 MB	60 MB	1
5	File Info	file_info.cpp	Interactive	2	90 MB	20 MB	1
6	Notepad	notepad.cpp	Interactive	2	150 MB	50 MB	1
7	Calculator	calculator.cpp	Interactive	2	100 MB	20 MB	1
8	Digital Clock	clock.cpp	Auto-running	2	50 MB	10 MB	1
9	System Info	system_info.cpp	Interactive	2	80 MB	10 MB	1
10	Snake Game	snake.cpp	Interactive	2	250 MB	40 MB	1
11	Minesweeper	minesweeper.cpp	Interactive	2	220 MB	40 MB	1
12	Music Player	music_player.cpp	Background	3	120 MB	30 MB	1
13	Download Simulator	download_simulator.cpp	Background	3	180 MB	200 MB	1
14	Task Manager	task_manager.cpp	System	1	100 MB	20 MB	1
15	Process Killer	process_killer.cpp	Kernel	1	100 MB	20 MB	1

8. Ready Queue Design

The multilevel queue uses three priority levels with distinct scheduling algorithms:

```
Priority 1 — System Queue      [FCFS]
Priority 2 — Interactive Queue [Round Robin, 8 s quantum]
Priority 3 — Background Queue  [FCFS]
```

Aging logic applied before each dispatch:

```
increment waiting time for all queued processes
if waiting_time >= AGING_THRESHOLD:
    improve numeric priority
    move process to the appropriate higher-priority queue
```

This design satisfies the manual requirement for multilevel queues with different algorithms at different levels and prevents indefinite starvation of low-priority processes.

9. Resource Allocation Strategy

Resources are checked by the kernel before a child process is permitted to run. The allocation sequence is:

```
Child Process  →  IPCResourceRequest (pipe)  →  Kernel

Kernel checks RAM, HDD, CPU availability:
  if available:
    allocate counters
    allocate RAM block (first-fit)
    create PCB entry
    add to ready queue
    send granted response
  else:
    send denied response
    child exits
```

The number of concurrently active tasks is therefore bounded by available RAM, hard drive space, and CPU core count. All resources are released and free blocks are coalesced when a process terminates.

10. User Interface

The terminal UI is standardized around an Ubuntu aesthetic using ANSI color codes via console_colors.h:

- Royal blue is the primary accent color for headers and section titles.
- Neutral gray and white are used for base interface text.
- Green, yellow, and red are reserved for success, warning, and error states respectively.
- Fixed-width tables use plain state text to preserve column alignment.
- The terminal launcher prefers GNOME Terminal on Ubuntu and falls back to x-terminal-emulator or xfce4-terminal.

In addition to the terminal menus, the SFML-based scheduler_terminal provides a graphical window that visualizes scheduler events in real time by reading data/scheduler_event_log.txt.

11. User Mode and Kernel Mode

AMS OS starts in User Mode. User Mode allows viewing normal system state and launching tasks. Kernel Mode is protected by password authentication (default password: admin) and unlocks privileged actions:

- View the full system log (data/system_log.txt)
- Force-kill a process
- Remove PCB entries
- Run resource allocation tests
- Run deadlock detection
- Manually update process state

12. Interrupt and Process Control

AMS OS supports the following manual process control operations:

Control	Menu Option	Effect
Minimize	17	Sends SIGSTOP, removes from ready queue, keeps resources allocated
Resume	18	Moves MINIMIZED or BLOCKED process back to READY
Close	21	Terminates process and releases all resources
Switch	22	Shows and resumes a selected process when applicable
Input interrupt	26	Moves task to BLOCKED state
Complete interrupt	27	Moves BLOCKED task back to READY
Kernel kill	16	Force-terminates a task (Kernel Mode only)

13. Manual Compliance Matrix

Requirement	Status	Implementation Detail
OS has its own name	✓ Complete	AMS OS displayed on boot screen
Boot loading with sleep	✓ Complete	bootScreen() uses sleep(1) with animated dots
User provides RAM, HDD, cores	✓ Complete	CLI args ./OS <RAM_GB> <HDD_GB> <CORES>
Resource-managed processes	✓ Complete	Resource Manager checks RAM/HDD/CPU before grant
Minimum 15 tasks	✓ Complete	15 task executables registered in catalog
Separate code file per task	✓ Complete	One .cpp source file per task
Process creation via fork/exec	✓ Complete	fork() and exec() used for every task launch
IPC resource message	✓ Complete	Child sends IPCResourceRequest through anonymous pipe
Deny if resources unavailable	✓ Complete	Parent sends denied response; child exits
Separate terminal per task	✓ Complete	GNOME Terminal / fallback terminal launcher
Close or minimize task	✓ Complete	Menu options 17, 18, 21, 22
Multitasking	✓ Complete	PCB table, ready queues, scheduler, separate processes
User Mode and Kernel Mode	✓ Complete	OSMode enum with password-protected Kernel Mode
Hardware access in Kernel Mode	✓ Complete	Kernel Mode process kill and resource tools
Time auto-start task	✓ Complete	Digital Clock auto-starts after boot
Instruction guide	✓ Complete	Menu option 23
RAM location and size in PCB	✓ Complete	Block startAddress/endAddress stored in PCB
Ready queue scheduling	✓ Complete	ReadyQueueManager and Scheduler modules
Multilevel queue scheduling	✓ Complete	System, Interactive, Background queues
Different algorithms per queue	✓ Complete	FCFS and Round Robin
Interrupt moves task to blocked	✓ Complete	Menu option 26
Resume after interrupt	✓ Complete	Menu option 27
Process lifecycle removal	✓ Complete	Scheduler and close/kill release resources

Requirement	Status	Implementation Detail
Data saved to virtual disk	✓ Complete	Virtual disk under data/virtual_disk/
Threads	✓ Complete	Resource monitor background thread
Semaphore	✓ Complete	SimpleSemaphore class
Mutex	✓ Complete	CPU core pool mutex
Condition variable	✓ Complete	Ready queue notification
Priority with aging	✓ Complete	ReadyQueueManager::applyAging()
Deadlock detection	✓ Complete	Wait-for graph DFS cycle detection
Required deadlock message	✓ Complete	Displays "Deadlock detected among processes"
System log file	✓ Complete	data/system_log.txt
Graphics library bonus	✓ Complete	SFML scheduler_terminal graphical event viewer

14. Build Configuration

The Makefile supports standard Linux build practices:

Target	Purpose
make	Build kernel OS executable and all task executables into build/
make run	Build and run with recommended resources (2 GB RAM, 256 GB HDD, 8 cores)
make check	Verify compiler and terminal emulator availability
make install	Run prerequisite checks, then install under /usr/local
make uninstall	Remove installed files
make clean	Remove generated binaries and runtime log
make distclean	Remove generated build/ and virtual disk directories
make help	Show usage summary

The install target creates an ams-os launcher that changes into the installed application directory before executing the kernel so that relative task paths (./build/) continue to work after installation.

15. Testing Plan

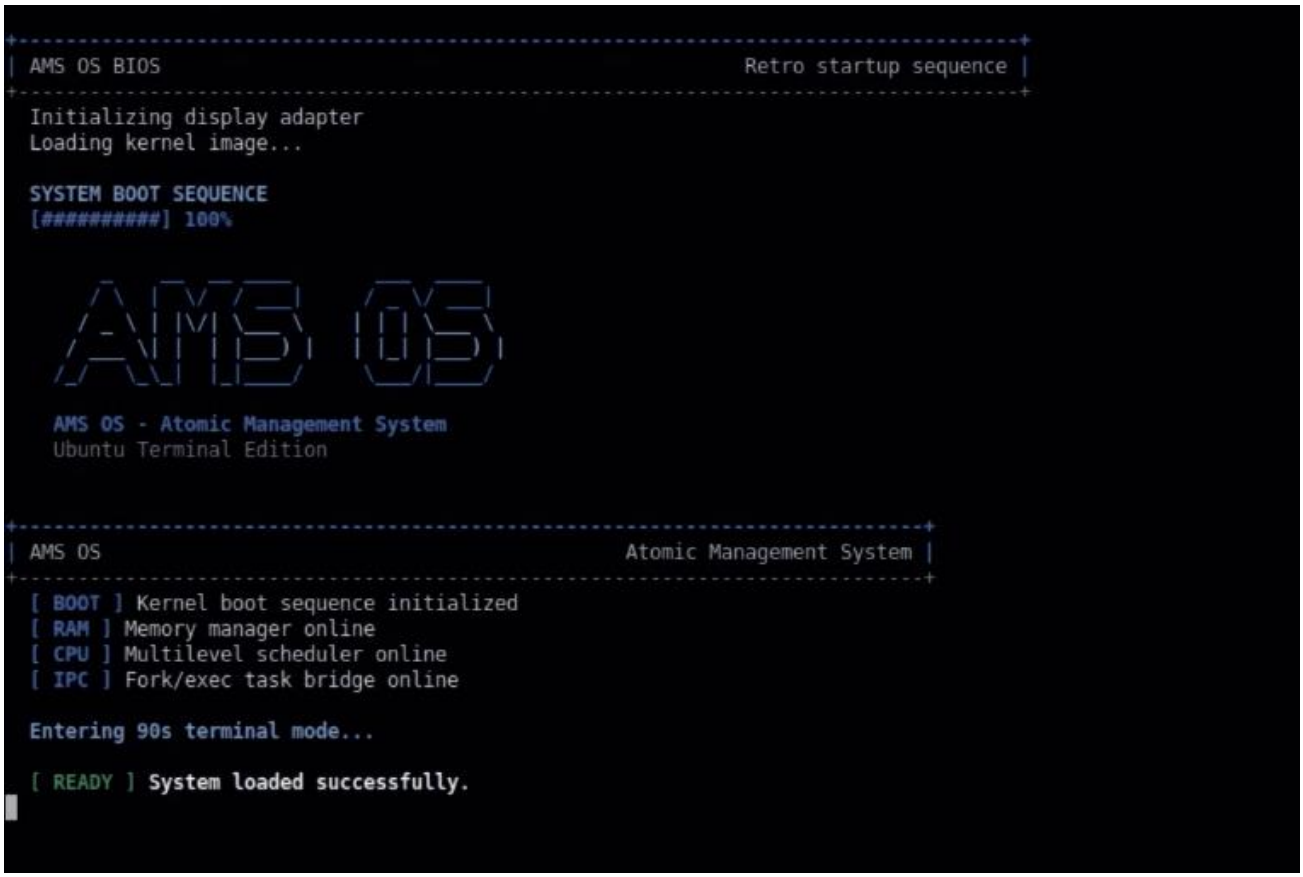
Recommended test sequence for academic demonstration:

- Run make check to verify toolchain availability.
- Run make to compile all modules and tasks.
- Start ./OS 2 256 8 and confirm the boot name and loading animation.
- Confirm Digital Clock auto-starts after boot.
- Launch Calculator, Notepad, Music Player, and File Copy.
- Run the scheduler and observe context switch output.
- Open the PCB table and verify PID, state, priority, queue label, and RAM block.
- Open Ready Queues and verify multilevel assignment.
- Minimize an interactive task (option 17) and resume it (option 18).
- Trigger an input interrupt (option 26) and a complete interrupt (option 27).
- Enter Kernel Mode with password admin.
- View data/system_log.txt through menu option 15.
- Run deadlock detection with two active processes.
- Launch the SFML scheduler terminal and verify the graphical event log.
- Close all tasks and verify resources are released.
- Shutdown and confirm cleanup.

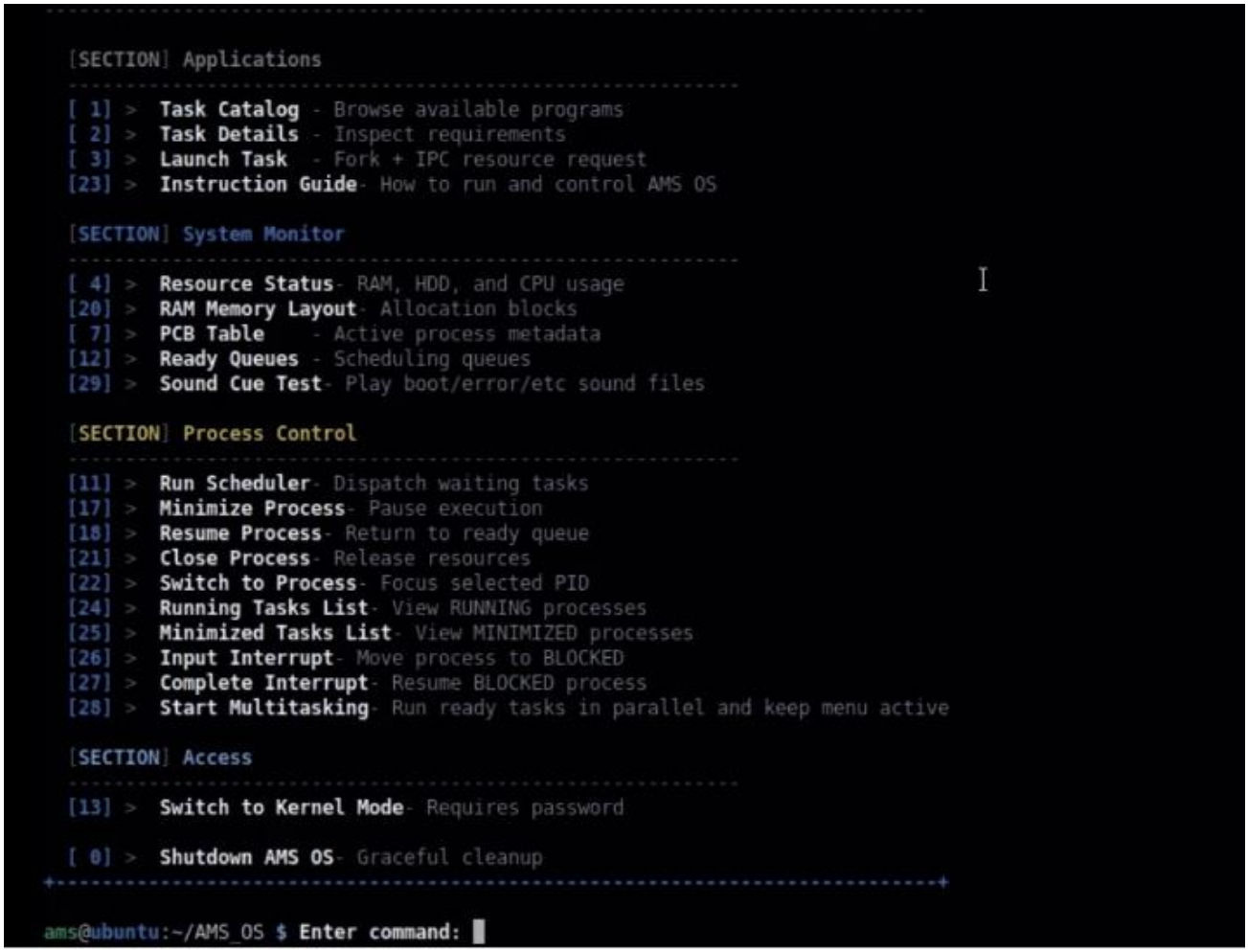
16. Known Limits

- AMS OS is a simulator; it does not manage real hardware resources.
- Process execution depends on Ubuntu / POSIX APIs and is not portable to non-Linux platforms without replacing the process model.
- Some terminal emulators handle process groups differently; GNOME Terminal is recommended for the cleanest Ubuntu experience.
- The SFML GUI (scheduler_terminal) requires libsfml-dev to be installed separately before building.

17. ScreenShots



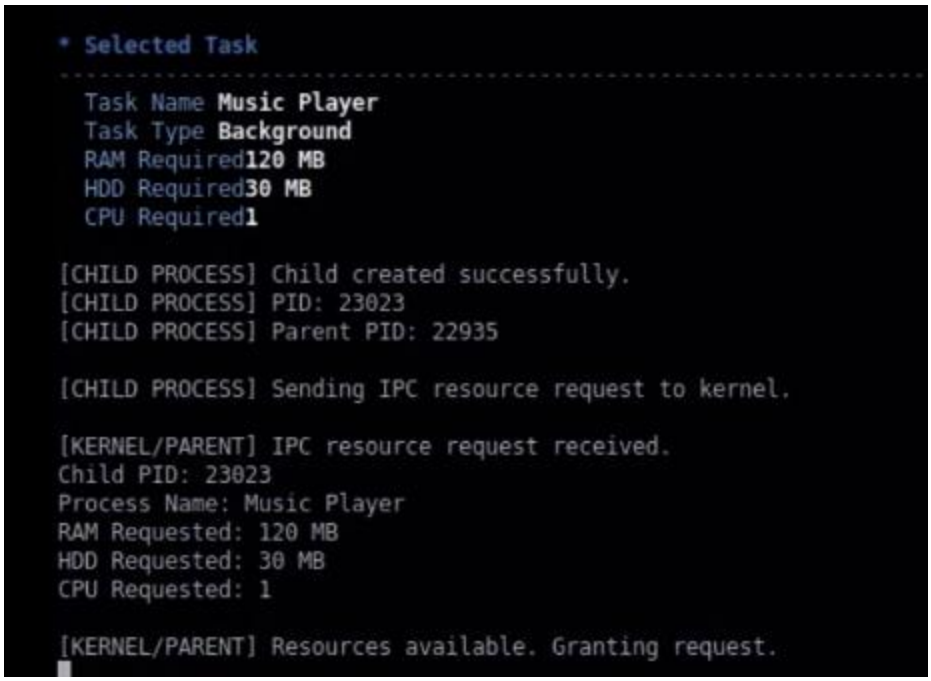
Booting


[SECTION] Applications
[1] > Task Catalog - Browse available programs
[2] > Task Details - Inspect requirements
[3] > Launch Task - Fork + IPC resource request
[23] > Instruction Guide- How to run and control AMS OS
[SECTION] System Monitor
[4] > Resource Status- RAM, HDD, and CPU usage
[20] > RAM Memory Layout- Allocation blocks
[7] > PCB Table - Active process metadata
[12] > Ready Queues - Scheduling queues
[29] > Sound Cue Test- Play boot/error/etc sound files
[SECTION] Process Control
[11] > Run Scheduler- Dispatch waiting tasks
[17] > Minimize Process- Pause execution
[18] > Resume Process- Return to ready queue
[21] > Close Process- Release resources
[22] > Switch to Process- Focus selected PID
[24] > Running Tasks List- View RUNNING processes
[25] > Minimized Tasks List- View MINIMIZED processes
[26] > Input Interrupt- Move process to BLOCKED
[27] > Complete Interrupt- Resume BLOCKED process
[28] > Start Multitasking- Run ready tasks in parallel and keep menu active
[SECTION] Access
[13] > Switch to Kernel Mode- Requires password
[0] > Shutdown AMS OS- Graceful cleanup
ams@ubuntu:~/AMS_OS $ Enter command:

Main menu

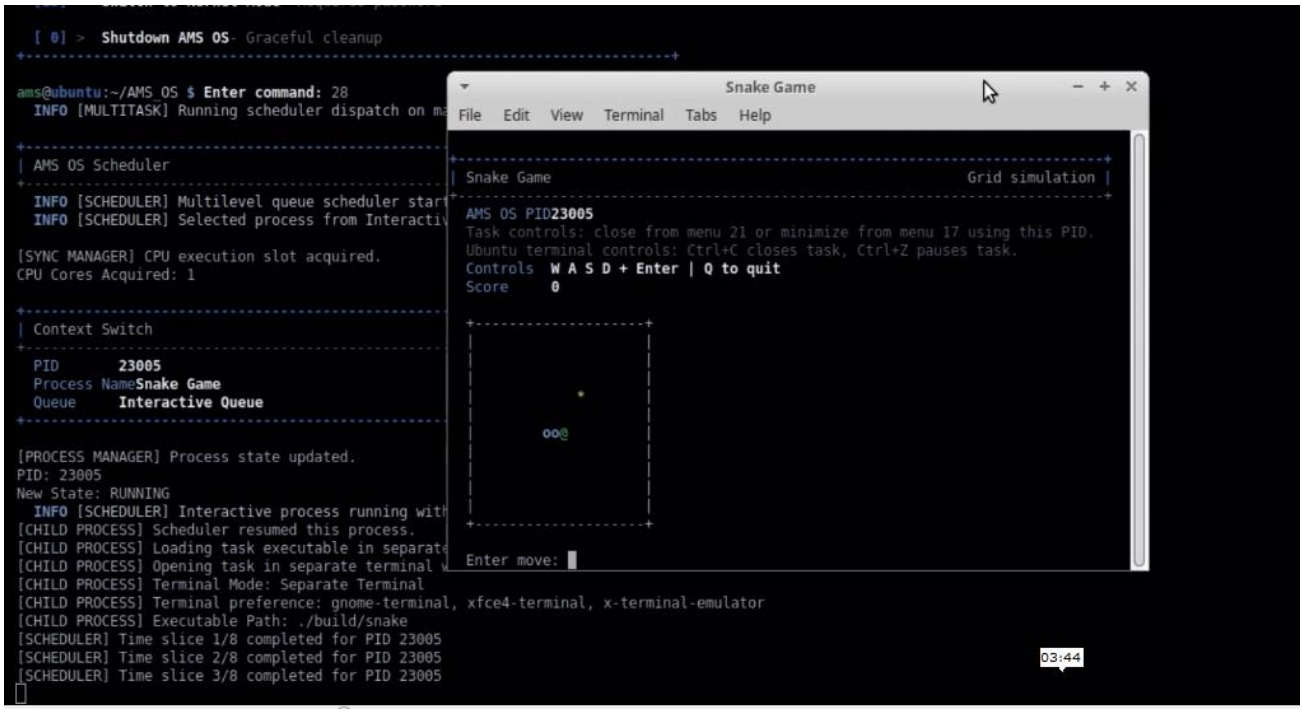

> Task 10 - Snake Game [Interactive] [P2]
Resources: RAM 250 MB | HDD 40 MB | CPU 1 core(s)
Description: Simple text-based snake game.
Exec: ./build/snake
> Task 11 - Minesweeper [Interactive] [P2]
Resources: RAM 220 MB | HDD 40 MB | CPU 1 core(s)
Description: Simple text-based minesweeper game.
Exec: ./build/minesweeper
> Task 12 - Music Player [Background] [P3]
Resources: RAM 120 MB | HDD 30 MB | CPU 1 core(s)
Description: Background music player simulation using beep output.
Exec: ./build/music_player
> Task 13 - Download Simulator [Background] [P3]
Resources: RAM 180 MB | HDD 200 MB | CPU 1 core(s)
Description: Simulates a background file download.
Exec: ./build/download_simulator
> Task 14 - Task Manager [System] [P1]
Resources: RAM 100 MB | HDD 20 MB | CPU 1 core(s)
Description: Task manager executable. Full process table is handled by kernel menu.
Exec: ./build/task_manager
> Task 15 - Process Killer [Kernel] [P1]
Resources: RAM 100 MB | HDD 20 MB | CPU 1 core(s)
Description: Kernel mode process killer information task.
Exec: ./build/process_killer
> Task 16 - Calendar [Auto] [P2]
Resources: RAM 50 MB | HDD 10 MB | CPU 1 core(s)
Description: Auto-running monthly calendar task.
Exec: ./build/calendar
Enter Task ID to launch:

Tasks Tab

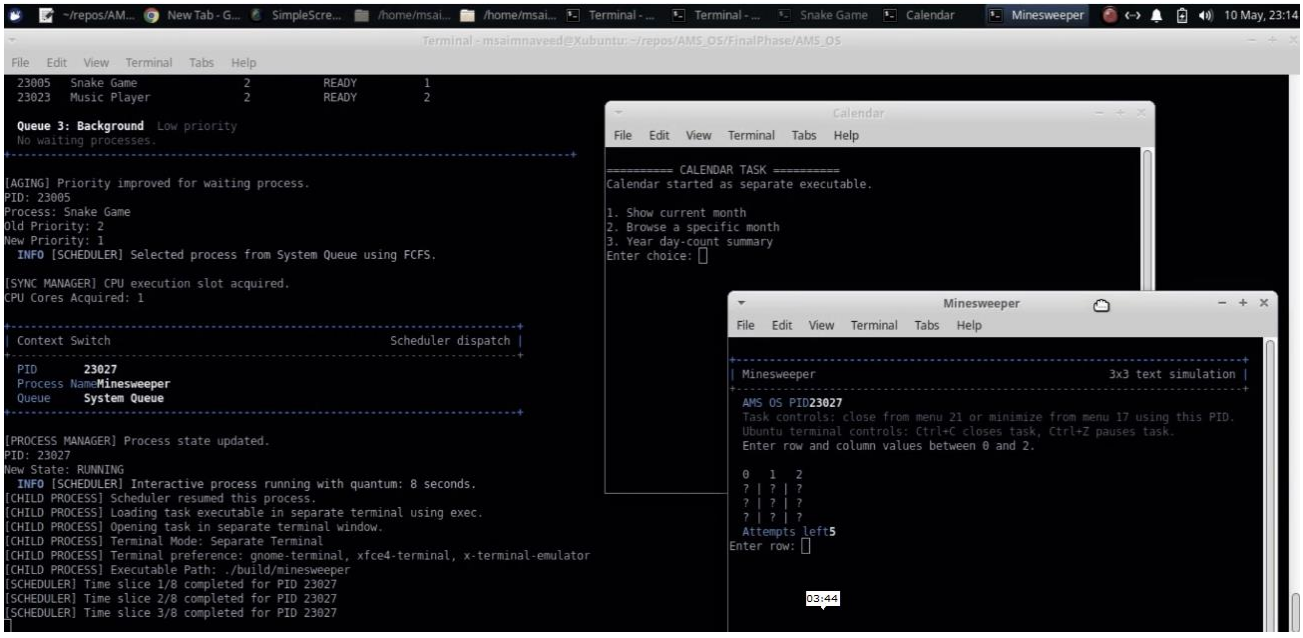


Parent-child communication

Ready-Queues



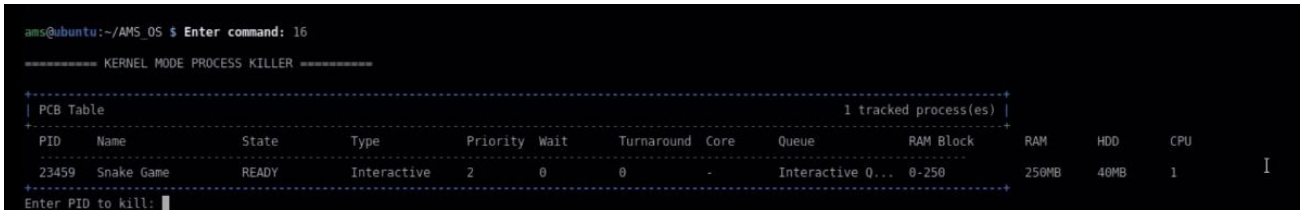
Separate Terminal Execution



MultiTaskin+Scheduling

Minimize Queue + Resuming

Kernel Authentication



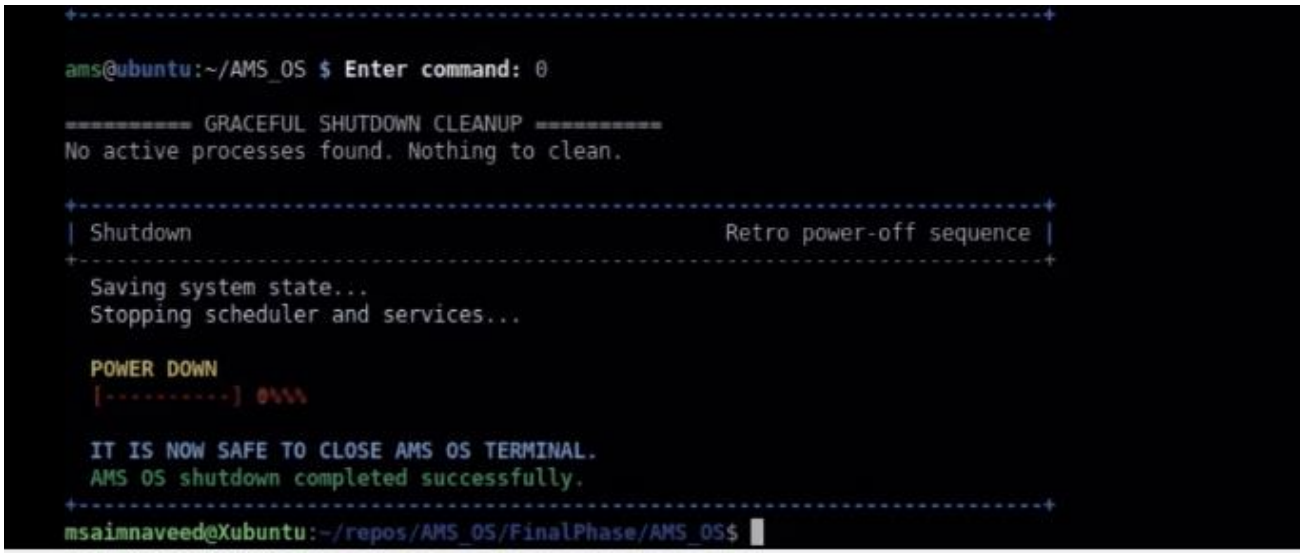
Process Kill by kernel

```
[Sun May 10 23:13:31 2026
] RESOURCE | RAM block assigned to PID 23027 | Minesweeper | Start: 420MB | End: 640MB
[Sun May 10 23:13:31 2026
] PROCESS | PID: 23027 | Minesweeper | PCB Created
[Sun May 10 23:13:31 2026
] PROCESS | PID: 23027 | Minesweeper | Added to Ready Queue
[Sun May 10 23:13:33 2026
] RESOURCE | Monitor Thread | Available RAM: 1408MB | Available HDD: 1928MB | Available CPU Cores: 0
[Sun May 10 23:13:35 2026
] PROCESS | PID: 0 | Download Simulator | Task selected for launch
[Sun May 10 23:13:36 2026
] RESOURCE | Resources denied for process Download Simulator
[Sun May 10 23:13:38 2026
] RESOURCE | Monitor Thread | Available RAM: 1408MB | Available HDD: 1928MB | Available CPU Cores: 0
[Sun May 10 23:13:43 2026
] RESOURCE | Monitor Thread | Available RAM: 1408MB | Available HDD: 1928MB | Available CPU Cores: 0
[Sun May 10 23:13:48 2026
] RESOURCE | Monitor Thread | Available RAM: 1408MB | Available HDD: 1928MB | Available CPU Cores: 0
[Sun May 10 23:13:50 2026
] SYSTEM | Scheduler started
[Sun May 10 23:13:50 2026
] SYSTEM | CPU execution slot assigned to PID 23005 | Process: Snake Game | Core: 2
[Sun May 10 23:13:50 2026
] PROCESS | PID: 23005 | Snake Game | Dispatched by scheduler
[Sun May 10 23:13:53 2026
] RESOURCE | Monitor Thread | Available RAM: 1408MB | Available HDD: 1928MB | Available CPU Cores: 0
[Sun May 10 23:13:58 2026
] RESOURCE | Monitor Thread | Available RAM: 1408MB | Available HDD: 1928MB | Available CPU Cores: 0
[Sun May 10 23:13:59 2026
] SYSTEM | CPU execution slot released after quantum expiry for PID 23005
[Sun May 10 23:13:59 2026
] PROCESS | PID: 23019 | Calendar | Aging applied, priority improved from 2 to 1
[Sun May 10 23:13:59 2026
] PROCESS | PID: 23027 | Minesweeper | Aging applied, priority improved from 2 to 1
[Sun May 10 23:13:59 2026
] PROCESS | PID: 23023 | Music Player | Aging applied, priority improved from 3 to 2
[Sun May 10 23:13:59 2026
] SYSTEM | CPU execution slot assigned to PID 23019 | Process: Calendar | Core: 3
[Sun May 10 23:13:59 2026
```

System log by kernel

```
+-------------------------------------------------------------------------------+
| Sound Cue Test                                        Play individual cue files |
+-------------------------------------------------------------------------------+
  1. boot.wav
  2. shutdown.wav
  3. error.wav
  4. granted.wav
  5. denied.wav
  6. minimize.wav
  7. resume.wav
  8. close.wav
  0. Back to main menu
+-------------------------------------------------------------------------------+
Select sound cue to play:
```

Sound test



PowerOff

17. Submission Checklist

- Complete source files at project root (kernel modules + task source files)
- 15 task executables defined in the Task Catalog
- Makefile with build, run, install, clean, and uninstall targets
- SFML scheduler terminal source (scheduler_terminal.cpp)
- Professional README.md
- Professional project report (this document)
- Manual compliance matrix (Section 13)
- System log support (data/system_log.txt)
- Virtual disk support (data/virtual_disk/)
- Ubuntu-focused terminal launcher
- No non-Linux source dependencies in kernel or task modules

18. Role-Based Responsibilities

If submitted as a group project, responsibilities can be mapped as follows:

Role	Responsibility
Kernel Developer (Saim)	Boot flow, menu system, fork/exec, IPC, interrupt and process controls
Scheduler Developer(Mohsin)	Ready queues, FCFS, Round Robin, aging, context switching

Role	Responsibility
Resource Developer(Saim)	RAM/HDD/CPU allocation and first-fit memory block layout
Task Developer(Ahmed)	Independent task executables (all 15 source files under tasks/)
M Saim Naveed	SFML scheduler_terminal graphical event viewer
Mohsin Ali Shah	README, project report, screenshots, and demo workflow
Ahmed Naeem	Ubuntu build/run testing and manual compliance verification

Review Note — *This report documents the complete AMS OS simulator design, implementation strategy, operating-system concepts, build configuration, manual compliance, and recommended demonstration workflow. For final submission, attach screenshots or demonstration-video frames covering: boot screen, task catalog, IPC resource grant, separate task terminal, PCB table, ready queues, resource layout, scheduler dispatch, SFML event viewer, Kernel Mode authentication, deadlock detection message, and shutdown cleanup.*